DAT321/DIT843: Quality Assurance and Testing

Questions about the exam contact:
Examiner: Francisco Gomes, francisco.gomes@cse.gu.se
Teachers: Francisco Gomes and Afonso Fontes

- Francisco will visit the exam halls around 15:00 and 16:30.
- Francisco is available for calls at the number: +46 73 856 4439

No aids are allowed for this exam.

The questions in this exam refer to the **ISO 25010:2011** that categorises four types of quality: process, internal, external and quality in use. The exam also covers the eight characteristics / attributes for product quality.

Important advice:

- Each question has points assigned shown in the square brackets. When the question is broken down into smaller sub-questions the part of the points for those specific sub-questions are also shown. Example: 1. [10 pts]. 1a. [2 pts] 1b. [8 pts].
- You must write correct, clear, readable, understandable, and unambiguous answers. If we cannot easily read what you are trying to write, then we cannot positively judge your knowledge or your learning of the topic.
- Motivate your answers (a simple statement of facts not answering the question is invalid). Whenever possible refer to modules, features and information about the software product, code or software development process related to the questions.
- Avoid circular definitions and vague / generic answers. Use unique aspects of each topic covered in your answers.
- Start each question on a new paper. Sort your answers in order (by question and sub-task) before handing them in. Write only on one side of the paper.
- Write your student code on each page and put the number of the question on every paper. The exam is anonymous, so do not leave any information that can reveal your identity.

The points and the corresponding grade are presented below (100 points in total):

- 00 – 49: U (Fail)
- 50 - 69: 3 (Pass)
- 70 - 84: 4 (Pass with credit)
- 85 - 100: 5 (Pass with distinction)

The date for the exam review session will be shared online via Canvas.

Question 1. [Total: 35 pts] You have been employed by Edu-Co, a company that develops an app for mobile devices, including phones, tablets, and laptops. This app offers educational games designed for children aged 5 to 10. Currently, it features several mini-games that cover basic concepts in Math, English, and History. Additionally, the system provides a separate app for parents. This app enables parents to set time limits on game usage and monitor their children's activity within the app, such as blocking specific exercises or locking the screen to prevent switching to other apps. Both apps are seamlessly integrated. Below are some features and characteristics of these apps:

- **F1:** The system shall allow customizable voice commands and touch gestures to accommodate different levels of motor skills among the target age group.
- **F2:** The team aims to add new mini-games every week to the software.
- **F3:** Children should be able to navigate between different games without parental help.
- **F4:** The parent monitoring app should be easy to insall in different mobile operating systems (iOS, Android) to ensure a broad user base.
- **F5:** The educational games app should be easily deployable across different devices, including phones, tablets, and laptops, without significant modifications.
- **F6:** Games within the app must load within three seconds on standard mobile devices to ensure a smooth user experience.
- **F7:** The parent monitoring app can remotely lock or unlock the screen of the childrens' devices via a 6-digit pincode defined by the parent.
- **F8:** Users in the app (children and parent) can exchange messages between them any time that the app is on.
- **F9:** If the children's device has a camera, the monitoring app should allow parents to login and access a live feed of the camera while their children is playing games.
- **F10:** The app must support synchronization of user progress and settings across multiple devices using the same user account, ensuring a seamless experience for users switching between devices.

Note: For the questions below, you are allowed to add features or decisions about the software development process (team size, technologies, etc.). However, any assumption related to your answer: (i) must be clearly state, and (ii) cannot contradict the points above.

a. **[7 pts]** One of your colleagues is tasked with evaluating the Security and Reliability aspects of the app. Consequently, select **other** two software product quality attributes and for each, provide: (i) a brief description of the quality attribute, and (ii) an explanation of its relevance to the software product described previously. When justifying your selections, refer to one or more of the points (F1 – F10) listed above.

[3pt] per quality attribute (+1 for defining, +1 for justifying, +1 for connecting the right features), and [+1pt] for the quality of the explanation.

Since Security and Reliability cannot be used, the remaining quality attributes are:

- Maintainability: Refers to the degree in which we can modify the software to release new version or fix defects. This is relevant because the team aims to release new games every week. Features: F2, F5.

- Compatibility: Refers to the degree in which the software product can exchange information and use that information with the hardware or other software. This is relevant because there are two parts of the app that need to communicate: the game and the parent monitoring component. Features: F7, F9, F10

- Portability: Refers to the degree in which the product can be deployed in different devices. This is relevant, because users should be able to install and run the software in a variety of hardware devices, such as phones, tablets. Features: F4, F5, F9

- Usability: Refers to the degree in which the user can complete tasks and exercise the functionalities of the system with ease. This is relevant because the app should cover a variety of users in different age groups along with their parents. Features: F1, F3

- Performance efficiency: Refers to the amount of resources used by product in different states or conditions. This is relevant because the app has different features that rely on time constraints, as well as memory resources to provide a positive gaming experience in real-time. Features: F6, F8, F9, F10

b. [8 pts] For each of the quality attribute chosen above create two quality scenarios for Edu-Co that includes: (i) source, (ii) stimulus, (iii) artefact, (iv) environment, (v) response, and (vi) response measure.

Some examples are shown below [4 pt] for each correct quality scenario:

Performance efficiency:

- Source: Game users
- Stimulus: Multiple users accessing the game simultaneously.
- Artefact: Server and database
- Environment: Cloud-based hosting environment, Peak usage during a school break.
- Response: System handles the load without significant performance degradation.
- Response Measure: Response times increase by no more than 10% of the average load conditions.

Portability:

- Source: Parent attempting to use the monitoring app.
- Stimulus: Download and installation on a new laptop.
- Artefact: Laptop client of the app.
- Environment: Various operating systems are installed in the laptop.
- Response: App installs and runs successfully on a chosen operating system.
- Response Measure: There are no crashes or interruptions when installing and operating on the laptop with latest OS versions.

Compatibility:

- Source: New user (child) using the game
- Stimulus: Accessing the game from an older version of iOS.
- Artefact: Mobile interface of Edu-Co
- Environment: User's personal iPhone with internet access. The iPhone has older version of iOS installed.
- Response: Platform maintains functionality.
- Response Measure: 95% of the features work as intended on mobile devices up to two versions behind the current release.

c. **[10 pts]** Explain the difference between internal and external quality. Choose two internal quality measures and, for each, explain (ii) what type of information that measure reveals about the system.

[4 pts] The difference is that: While internal quality assesses the quality of software artefacts, such as code, design, requirements, etc, external quality assesses the quality of the product when under execution (e.g., runtime).

[3 pts each] Some examples below of internal quality measures. A tip is to focus on code-related measures:

- LOC: Lines of Code is a measure that conveys size of the product.
- Fan-in, and fan-out are measures that conveys information about dependency or modularity of functions, or components in the system
- Code churn is a measure that conveys a difference between code added or deleted over time.
- Number of commits is a measure that shows the number of code changes made and integrated into a code repository.
- Number of documented methods is a measure to convey the readability of different functions of the code.

d. **[10 pts]** Ed-Co wants to promote sustainability in the product that we are creating. Using the Sustainability Awareness Framework, you must (i) **choose or suggest two functionalities** for the product and explain the (ii) immediate, (iii) enabling and (iv) structural effects of those features. You must also indicate the corresponding dimension of sustainability in which those effects.

[5 pts for each suggestion]. The goal is to (i) have a clear connection between the immediate, enabling and structural effect, and (ii) indicate the correct sustainability dimension (Social, Individual, Environment, Technical and Economical).

Suggestion 1: Help parents and children reduce screen time.
Immediate: Add a feature to track and show history of screen time used by the parent or child, along with options to lock the screen after a threshold (Technical).
Enabling: Establishes responsible usage patterns for children, leading to better social interactions and time management skills outside of the app's environment (Individual).
Structural: Contributes to reducing energy consumption as devices can be locked remotely when not in use, ensuring they are not left running unnecessarily and saving on energy (environmental).

Suggestion 2: Include educational resources on sustainability within the games.
Immediate: Integrate mini-games that teach economic and environmental sustainability concepts (Educational).
Enabling: Fosters awareness and understanding of sustainability, encouraging eco-friendly behavior in the household (Social).
Structural: Cultivates a generation that is more informed and proactive about sustainability challenges and practices (Environmental).

Question 2. [Total: 35 pts] You are asked to test the program specified below. You are also given the list of functions below to assist your process of writing test cases, or properties for your software. Based on your knowledge about Software Testing Techniques, answer the questions below.

Specification: The function `arrayMultiplier(inputArray, multiplier)` takes an array of integers (`inputArray`) and a single integer (`multiplier`) as inputs. It returns a new array where each element in inputArray has been multiplied by multiplier. If inputArray is empty, it returns an empty array. If inputArray is null, the method throws an Exception. Some examples of test cases are shown below:

Note: Since the implementation is done in Java, you can assume that: (i) there cannot be null elements inside the array of integers, (ii) the first index of the array is 0.

T1	`arrayMultiplier([1, 2, 3], 2) → [2, 4, 6]`
T2	`arrayMultiplier([-1, 0, 1], -1) → [1, 0, -1]`

T3	`arrayMultiplier([], 5) → []`
T4	`arrayMultiplier([5, 10, 15], 0) → [0, 0, 0]`
T5	`arrayMultiplier([100], 1) → [100]`
T6	`arrayMultiplier(null, 2) → Exception`

List of methods that can be used to write your properties:

`sort(int[]): int[]`	Returns the array with its values sorted in an ascending order.
`reverse(int[]):int[]`	Returns the array where its values are in reversed positions.
`length(int[]): int`	Returns the number of elements in the array.
`sum(int[]): int`	Returns the sum of elements in the array.
`max(int[]):int`	Returns the highest value inside the array
`min(int[]):int`	Returns the lowest value inside the array

a. [15 pts] Create exactly five properties to test the `arrayMultiplier` function. You can write the properties using natural language, mathematical notation, or pseudocode. Feel free to use the support functions listed above. Your properties will be evaluated based on the diversity of their input and output.

[+3pt each property], we expect to see varied properties. Below are some examples

Property 1: For any non-null inputArray and any multiplier: sum(inputArray) * multiplier = sum(arrayMultiplier(inputArray, multiplier))

Property 2: For any non-null inputArray and any multiplier > 0: max(inputArray) * multiplier = max(arrayMultiplier(inputArray, multiplier))

Property 3: For any non-null inputArray and any multiplier = 0: max(arrayMultiplier(inputArray, multiplier)) == min(arrayMultiplier(inputArray, multiplier)) == 0

Property 4: For any multiplier and inputArray == null, the system will throw an exception.

Property 5: For any non-null inputArray, and multiplier = 1, arrayMultiplier(inputArray, multiplier) == inputArray

Property 6: For any non-null inputArray and multiplier, length(arrayMultiplier(inputArray, multiplier)) == length(inputArray).

b. [12 pts] Using your knowledge of Search-based Software Testing, (i) explain the difference between a Meta-Heuristic and a Fitness Function, and (ii) provide an example of how test artefacts or concepts can be used with meta-heuristics and fitness functions. (iii) You must justify your example in relation to the generation of test cases that can reveal faults.

[5 pts] i) The difference is that Meta-heuristics are used to sample candidate solutions, whereas Fitness functions evaluate "how good" the solution is in relation to the optimisation goal.

[5 pts] ii) An example is to use a genetic algorithm as a meta-heuristic and represent the solution as a test suite, and code coverage as the fitness function. Other examples are hillclimber for a test suite as a candidate, or mutation / requirements coverage as fitness functions.

[2 pts] iii) Therefore, the search algorithm can evaluate multiple test suites (candidate solutions) in relation to a test adequacy criterion that is able to convey the fault capability of a test suite, such as branch coverage (e.g., to ensure we cover different values in a branch), mutation coverage (e.g., to ensure we cover different types of mistakes a developer can make when coding), or requirements coverage (e.g., to ensure we have tests that exercise important or a diversity of features of the system).

c. [8 pts] Using the example of test cases above, identify two boundaries. Explain (i) what the equivalent classes of those corresponding boundaries are. You can create additional test cases to better explain your boundaries and partition classes.

[4pt each] To identify the boundaries, find test cases that have similar inputs but different behaviours.

T5 belongs to the class of arrays of length 1.
T6 belongs to the class of null arrays.
T1 belongs to the class of non-null arrays and positive multipliers.
T2 belongs to the class of non-null arrays and negative multipliers.

Some examples of boundaries are between T5 and T6, or T1 and T2.

Question 3. [Total: 30 pts] Using your knowledge of test design and mutation testing answer the questions below. Your answers should be connected to the testing of the specification and code below.

Specification: The function `mergeInMiddle(listA,listB)` combines the elements of two lists in a single list. The method receives as input two lists of integers: listA, listB. The

output is a single list where all elements of listB are inserted in the middle index of listA. If any of the lists is null, the method will throw an Exception. See the code below:

```
1.public static int[] mergeInMiddle(int[] numbersA,int[] numbersB)throws Exception {
2.
3.  if(numbersA == null || numbersB == null) {
4.      throw new Exception("Arrays cannot be null.");
5.  }
6.
7.  int[] mergedArray = new int[numbersA.length + numbersB.length];
8.  int middleIndexA = (numbersA.length / 2);
9.
10.  for(int i = 0; i < middleIndexA; i++) {
11.      mergedArray[i] = numbersA[i];
12.  }
13.
14.  for(int i = 0; i < numbersB.length; i++) {
15.      mergedArray[middleIndexA + i] = numbersB[i];
16.  }
17.
18.  for(int i = middleIndexA; i < numbersA.length; i++) {
19.      mergedArray[numbersB.length + i] = numbersA[i];
20.  }
21.
22.  return mergedArray;
23. }
```

a. **[10 pts]** Create at least three equivalent partition classes for the `mergeInMiddle` function.

[+3 pts each partition], and [+1 pt] for the quality of the explanations and variety of classes. When defining the partition classes, mention the set of inputs and the corresponding similar behaviour triggered. Some examples:

Partition 1: One or both of the lists are null. Behaviour: Throws exception
Partition 2: One or both of the lists are empty but not null. Behaviour is to return the non-empty list, or an empty list in case both are null.
Partition 3: numbersA has only one element, numbersB is non-empty. Behaviour: The last element of the resulting list is the element in numbersA.

b. **[10 pts]** Using the partition classes that you defined above, create test cases that reveal three boundaries. You must indicate the corresponding equivalent partition classes of each boundary listed.

[+3 pts each partition], and [+1 pt] for the quality and variety of chosen test cases. When defining boundaries, there should be at least a pair of tests that should have similar input values, and dissimilar output values (e.g., in terms of array length, or composition of elements). Some examples:

Boundary between P1 and P2:
TC1: Input: numbersA = [1], numbersB = null. Output: Exception
TC2: Input: numbersA = [1], numbersB = []. Output: [1]

Boundary between P1 and P3:
TC1: Input: numbersA = [1], numbersB = null. Output: Exception
TC3: Input: numbersA = [1], numbersB = [2]. Output: [2, 1]

Boundary between P2 and P3:
TC3: Input: numbersA = [1], numbersB = [] . Output: [1]
TC4: Input: numbersA = [1], numbersB = [2]. Output: [2, 1]

c. **[10 pts]** For the code above, (i) create two mutants for the code above using different mutation operators. You only need to write the specific line of code that you mutated. (ii) Specify which mutation operators you used, and (iii) sow an example of two test cases that would kill each mutant.

[5 pts per mutant]. The mutant must correspond to the correct operator [2pt], and the test case must kill the mutant [3 pt]. Note that the code that does not compile is not a mutant [0 pt].

Conditional operator replacement: Line 3
Mutant: **if**(numbersA == **null** **&&** numbersB == **null**) {… }
Test: numbersA = null, numbersB = []. (this will not throw the exception as expected).

Arithmetic Operator Replacement: Line 7
Mutant: **int**[] mergedArray = **new int**[numbersA.**length** - numbersB.**length**];
Test: numbersA = [1], numbersB = [1, 2, 3]. (this will trigger an exception in Java, when it should not)

Relational Operator Replacement: Line 10
Mutant: **for**(**int** i = 0; i ≥ middleIndexA; i++) {
Test: numbersA = [1, 2, 3], numbersB = [2]. (the first half of numbersA will not be in the final array).